Angry Poutine — Corrigé

Oct 30, 2026 · François Paquin, CISSP

Corrigé détaillé de la fiche d'exercice : réponses attendues, commandes et filtres expliqués. Toutes les heures sont en UTC

Étape 1 — Les flux (nfpcapd et nfdump)

Conclusion attendue : le poste 10.9.10.102 télécharge un fichier de 293 Ko en HTTP à 23:17:27, puis contacte 94.158.245.52:443 à intervalles réguliers jusqu'à la fin de la capture.

Repères de syntaxe nfdump

- `-R ./flux` lit tous les fichiers du répertoire ; `-r fichier` en lit un seul.
- `-q` retire l'en-tête et le résumé ; `-N` affiche des nombres entiers (sinon « 2.0 M »).
- `-s <clé>/<mesure>` fait un top N (`-n 15`) ; `-O tstart` trie par heure de début.
- `-B` regroupe les deux sens d'une connexion en un seul flux bidirectionnel et place le client en source. À préférer à `-b`, qui garde comme source le premier paquet vu, parfois le serveur.
- Le filtre, entre apostrophes à la fin, a une syntaxe proche de BPF : `host`, `src host`, `dst port`, `net`, `proto tcp`, `flags S`, combinés par `and`, `or`, `not`.
- `-o "fmt:..."` choisit les colonnes : `%ts` début, `%td` durée, `%pr` protocole, `%sap`/`%dap` IP:port, `%flg` drapeaux TCP, `%pkt` paquets, `%byt` octets.

A. Extraction

Q1. 16 fichiers (capture de 77 min découpée en tranches de 5 min).

```
export TZ=UTC
mkdir flux
nfpcapd -r angry-poutine.pcap -w ./flux
ls flux
```

`-r` lit un pcap au lieu d'une interface, `-w` donne le répertoire de sortie (alias accepté : `-l`). nfpcapd imite un collecteur NetFlow : il fait une rotation des fichiers toutes les 300 secondes (option `-t`). Le nom `nfcapd.AAAAMMJJhhmm` suit le fuseau horaire au moment de l'extraction, d'où l'importance de `TZ=UTC`.

Q2. 706 flux, 8 959 paquets, 5 875 491 octets. Premier flux : 2021-09-10 23:15:07 ; dernier : 2021-09-11 00:31:57. Durée : environ 1 h 17 min.

```
nfdump -R ./flux -I
date -u -d @1631315707
date -u -d @1631320317
```

`-I` affiche les statistiques globales ; les heures `First` et `Last` sont en secondes Unix (epoch), converties avec `date`.

Q3. 472 flux TCP, 233 UDP, 1 autre (IGMP). Même sortie `-I`, ou `nfdump -R ./flux -q -s proto`.

B. Qui est qui

Q4. 10.9.10.102, 10.9.10.9 et 10.9.10.103 (plus 10.9.10.255, l'adresse de diffusion, et 0.0.0.0 pendant le DHCP).

```
nfdump -R ./flux -q -o "fmt:%sa %da" 'net 10.9.10.0/24' | tr -s ' ' '\n' | grep '^10\.9\.10\.' | sort | uniq -c
```

On affiche seulement les IP source et destination, on met une adresse par ligne, on garde celles du LAN et on compte.

Q5. 10.9.10.9 est le contrôleur de domaine. Ports reçus : 53 DNS (112 flux), 445 SMB, 389 LDAP, 135 RPC (mappeur de points de terminaison) et 49667/49671 (RPC dynamique), 88 Kerberos, 123 NTP. Kerberos + LDAP + DNS au même endroit est la signature d'un contrôleur Active Directory.

```
nfdump -R ./flux -q -s dstport/flows 'dst host 10.9.10.9'
```

Q6. 10.9.10.102 : 703 des 706 flux (99,6 %) et 5,9 Mo. C'est le seul poste client actif de la capture.

```
nfdump -R ./flux -q -s ip/bytes -n 5
```

C. Destinations externes

Q7. En tête : 23.1.237.225 (2,1 Mo), 23.1.237.216 (1,1 Mo), 52.238.248.6 (448 Ko), 23.1.237.200 (312 Ko), **167.172.37.9 (307 Ko)**, **194.62.42.206 (297 Ko)**, **94.158.245.52 (198 Ko)**, puis des adresses Akamai et Microsoft. À l'ouverture de session, Windows contacte Windows Update, la télémétrie, Office, Edge et les services de licence : c'est le bruit de fond normal.

```
nfdump -R ./flux -q -s ip/bytes -n 16
```

Le poste lui-même occupe la première ligne, d'où `-n 16` pour en voir 15 autres.

Q8. 194.62.42.206:80, début 23:17:27.500, 293 719 octets reçus en 212 paquets et 2,2 s. Il ne s'agit pas d'un CDN, la connexion arrive 30 s après l'ouverture de session et elle ouvre une séquence suspecte : probablement le téléchargement d'un exécutable.

```
nfdump -R ./flux -q -O tstart -B -o "fmt:%ts %td %sap <-> %dap %ipkt %opkt %ibyt %obyt" 'port 80'
```

Avec `-B`, la source est le poste : `%ibyt` = octets envoyés par le poste, `%obyt` = octets reçus. On utilise un format explicite parce que la sortie bidirectionnelle par défaut n'a pas d'en-tête avec `-q` et prête à confusion sur l'ordre des colonnes. Point pédagogique : le volume seul ne suffit pas, les transferts Akamai sont dix fois plus gros. C'est le contexte (propriétaire de l'IP, moment, suite d'événements) qui rend celui-ci suspect.

Défi (flux DNS) : une requête DNS de 65 octets part à 23:17:27.020, 0,5 s avant. Une requête A sans EDNS fait 44 octets + la longueur encodée du nom (nom + 2). 65 − 44 = 21, donc un nom de 19 caractères : c'est exactement `simpsonsavingss.com`, confirmé à l'étape 2. Deuxième partie du défi : la requête de 59 octets de 23:17:31.802 donne un nom de 13 caractères, juste avant la connexion à 167.172.37.9. Tentant de les relier… mais c'était `microsoft.com`, et 167.172.37.9 n'apparaît dans aucune réponse DNS. Leçon : une coïncidence de temps n'est pas une preuve, et c'est à l'étape suivante de trancher.

Q9. 167.172.37.9:443 (hébergeur infonuagique DigitalOcean), début 23:17:31.848, 300 565 octets reçus en 226 paquets et 2,6 s. Volume semblable : probablement un deuxième fichier (la charge utile suivante).

```
nfdump -R ./flux -q -B -o "fmt:%ts %td %sap <-> %dap %ipkt %opkt %ibyt %obyt" 'host 167.172.37.9'
```

Q10. 94.158.245.52, port 443 : 110 flux unidirectionnels de 23:17:36 à 00:30:22.

```
nfdump -R ./flux -q -s ip/flows -n 10
nfdump -R ./flux -q -O tstart 'host 94.158.245.52'
```

D. Comportement de 94.158.245.52

Q11. 33 sessions. Elles arrivent **par paires** : une session longue (1 à 8 min, maintenue ouverte) puis une courte (2 à 6 s) quelques secondes plus tard. Les paires reviennent toutes les 3 à 8 minutes (171 à 505 s) : régulier, mais avec une variation aléatoire (« jitter »).

```
nfdump -R ./flux -q -O tstart -o "fmt:%ts" 'src host 10.9.10.102 and dst host 94.158.245.52 and flags S' | awk '{split($2,t,":"); s=t[1]*3600+t[2]*60+t[3]; if(NR>1){d=s-p; if(d<0)d+=86400; printf "%s %s  +%ds\n",$1,$2,d} else print $1,$2; p=s}'
```

`flags S` ne garde que les flux qui contiennent un SYN, donc un par session TCP. Le script awk convertit l'heure en secondes et affiche l'écart avec la ligne précédente ; `+86400` gère le passage de minuit.

Q12. Le poste envoie 139 772 octets et en reçoit 57 961, soit environ 198 Ko en 73 minutes. Envoyer plus que recevoir est inhabituel pour de la navigation, mais le volume est bien trop faible pour une exfiltration massive : ce sont des rapports et des sondages.

```
nfdump -R ./flux -q -N -o "fmt:%sa %byt" 'host 94.158.245.52' | awk '{b[$1]+=$2} END{for(k in b) print k, b[k]}'
```

Q13. C'est du **beaconing** : le maliciel contacte régulièrement son serveur de commande et contrôle (C2) pour signaler qu'il est vivant et recevoir des ordres. Le jitter sert à échapper aux détections basées sur un intervalle fixe.

E. Chronologie et portée

Q14.

Heure (UTC)	Événement
23:16:55.749	Premier flux Kerberos vers le DC : ouverture de session
23:17:27.020	Requête DNS (19 caractères)
23:17:27.500	Téléchargement HTTP depuis 194.62.42.206
23:17:31.848	HTTPS vers 167.172.37.9
23:17:36.466	Premier contact avec 94.158.245.52

Délai téléchargement → premier contact C2 : 9 secondes.

```
nfdump -R ./flux -q -O tstart -t 2021/09/10.23:16:30-2021/09/10.23:18:00 -o "fmt:%ts %pr %sap -> %dap %flg %byt" 'src host 10.9.10.102'
```

`-t` limite à une fenêtre de temps (dans le fuseau de `TZ`).

Q15. Oui, une seule fois : 10.9.10.103, à 23:19:22, avec du NetBIOS (UDP 137/138) et une session TCP 139 de 16 s, environ 3 Ko. Pas de mouvement latéral (aucun fichier copié, aucune exécution à distance), mais l'échange tombe dans une rafale à 23:19:21–27 (34.117.59.81, SMB vers le DC, nouvelle paire C2) :

on peut soupçonner une reconnaissance automatique. Les étapes 2 et 3 le confirment : c'est un `net view` (énumération des postes du réseau). Accepter « reconnaissance possible » comme bonne réponse à ce stade.

```
nfdump -R ./flux -q -O tstart 'host 10.9.10.103'
```

F. Bilan

Q16. Exemple : « Après l'ouverture de session sur 10.9.10.102, le poste télécharge environ 290 Ko en HTTP depuis 194.62.42.206, puis 300 Ko en HTTPS depuis 167.172.37.9. Neuf secondes plus tard, il commence à communiquer avec 94.158.245.52 toutes les 3 à 8 minutes jusqu'à la fin de la capture. Hypothèse : infection par un chargeur qui a installé une porte dérobée avec un canal C2 actif. »

Q17. Les noms de domaine, l'URL complète, la nature et le hash du fichier, le nom d'hôte, le compte utilisateur, le User-Agent, le SNI et le certificat TLS, le contenu des échanges C2.

Étape 2 — Les paquets bruts (tcpdump)

Conclusion attendue : le poste DESKTOP-KKITB6Q, utilisé par hobart.gunnarsson, a téléchargé un exécutable Windows depuis `simpsonsavingss.com`. Les deux IP suivantes sont codées en dur dans le maliciel (aucune résolution DNS). Deux minutes après l'infection, le poste a énuméré les postes et les groupes du domaine.

Repères de syntaxe tcpdump

- `-r f` lit un fichier ; `-nn` ne traduit ni les IP ni les ports en noms (plus rapide, et on voit les vraies valeurs).
- `-v` / `-vv` décodent davantage de champs ; `-e` affiche l'en-tête Ethernet (adresses MAC).
- `-A` affiche le contenu en ASCII ; `-X` en hexadécimal + ASCII.
- `-tttt` affiche date et heure complètes (dans le fuseau de `TZ`, d'où `export TZ=UTC`).
- `-c N` s'arrête après N paquets ; `-w -` écrit les paquets bruts filtrés sur la sortie standard, pour les passer à `strings`.
- Le filtre BPF : `host`, `src`/`dst`, `port`, `tcp`/`udp`, et l'accès aux octets d'un en-tête avec `tcp[décalage:taille]`.

Particularité de tcpdump 4.99 : le décodage NetBIOS et SMB n'est plus compilé par défaut. tcpdump n'affiche donc ni les noms NetBIOS ni les commandes SMB. Pour lire ce trafic, on extrait les chaînes avec `strings`, en ASCII et en UTF-16 (`-e l`), l'encodage qu'utilise Windows.

A. Identité du poste

Q1. DESKTOP-KKITB6Q, dans l'option DHCP 12 (*Hostname*) du Discover et du Request. Le Request contient aussi l'option 81 (*FQDN*) : `DESKTOP-KKITB6Q.angrypoutine.com`.

```
tcpdump -nn -v -r angry-poutine.pcap 'udp port 67 or udp port 68'
```

Sortie (extrait) : `Hostname (12), length 15: "DESKTOP-KKITB6Q"`. Le DHCP occupe les quatre premiers paquets de la capture (23:15:07).

Q2. 00:4f:49:b1:e8:c3.

```
tcpdump -nn -e -c 3 -r angry-poutine.pcap 'src host 10.9.10.102'
```

`-e` ajoute l'en-tête de couche 2. Le DHCP l'affiche aussi : `Client-Ethernet-Address 00:4f:49:b1:e8:c3`.

Q3. Le serveur DHCP est 10.9.10.9 (`Server-ID (54)`), donc le contrôleur de domaine. Il attribue `Your-IP 10.9.10.102`, la passerelle 10.9.10.1, les serveurs DNS **10.9.10.9 et 8.8.8.8**, et le domaine `angrypoutine.com`.

```
tcpdump -nn -v -r angry-poutine.pcap 'udp src port 67'
```

On garde seulement les réponses du serveur (Offer et ACK). Le `^@` à la fin de `"angrypoutine.com^@"` est un octet nul de fin de chaîne. Point de discussion : un DNS public (8.8.8.8) en secours sur un poste de domaine contourne le filtrage DNS interne si le DC ne répond pas.

B. Le DNS

Q4. 48 noms différents. Presque tous sont Microsoft (mises à jour, Office, Store, télémétrie). Ceux qui sortent du lot : `simpsonsavingss.com`, `myexternalip.com` et `DESKTOP-MUYEYV7.angrypoutine.com`. `wpad.angrypoutine.com`, `gameplayapi.intel.com` et `update.googleapis.com` sont du bruit Windows ou logiciel normal.

```
tcpdump -nn -r angry-poutine.pcap 'src host 10.9.10.102 and udp dst port 53' | grep -oE ' A\? [^ ]+' | sort -u
```

tcpdump affiche une requête sous la forme `A? nom.`. L'espace avant le `A` dans le motif évite d'attraper les lignes `AAAA?`. `sort -u` retire les doublons ; ajouter `| wc -l` pour compter.

Q5. simpsonsavingss.com → 194.62.42.206. Requête à 23:17:27.020, réponse à 23:17:27.253.

```
tcpdump -nn -tttt -r angry-poutine.pcap 'udp src port 53' | grep '194.62.42.206'
```

Sortie : `2021-09-10 23:17:27.253809 IP 10.9.10.9.53 > 10.9.10.102.57667: 40513 1/0/0 A 194.62.42.206 (53)`. On cherche l'IP dans les **réponses** (port source 53). `1/0/0` = 1 réponse, 0 autorité, 0 enregistrement additionnel.

Q6. Non : ni 167.172.37.9 ni 94.158.245.52 n'apparaissent dans une réponse DNS. Leurs adresses sont inscrites directement dans le maliciel. Conséquence : un filtrage DNS n'aurait pas coupé ces connexions ; il faut bloquer les IP. (La requête de 23:17:31 juste avant 167.172.37.9 était `microsoft.com` : voir le défi de l'étape 1.)

```
tcpdump -nn -tttt -r angry-poutine.pcap 'udp src port 53' | grep -E '167\.172\.37\.9|94\.158\.245\.52'
```

Aucune ligne en sortie : c'est la réponse. Le point non échappé veut dire « n'importe quel caractère » en expression régulière, d'où `\.`.

C. Le téléchargement HTTP

Q7. Requête envoyée à 23:17:27.822 :

```
GET /bmdff/BhoHsCtZ/MLdmpfjaX/5uFG3Dz7yt/date1?BNLv65=pAAS HTTP/1.1
Accept: */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 10.0; WOW64; Trident/7.0; .NET4.0C; .NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.30729; .NET CLR 3.5.30729)
```

```
Host: simpsonsavingss.com
Connection: Keep-Alive
```

```
tcpdump -nn -A -r angry-poutine.pcap 'tcp dst port 80 and dst host 194.62.42.206' |
grep -aE 'GET |Host:|User-Agent:'
```

Avec `-A`, les octets des en-têtes IP et TCP s'affichent aussi en ASCII, collés devant `GET` : le motif ne commence donc pas par `^`. `-a` force `grep` à traiter la sortie comme du texte.

Q8. `Windows NT 10.0` → Windows 10 (Windows 11 n'existait pas encore). `WOW64` → un processus 32 bits sur un Windows 64 bits. `MSIE 7.0 ... Trident/7.0` → ce n'est pas un navigateur moderne, mais le composant de téléchargement de Windows utilisé par des scripts ou des applications. Indice que le téléchargement n'a pas été fait par l'utilisateur dans son navigateur.

Q9. Réponse à 23:17:28.562 :

```
HTTP/1.1 200 OK
Date: Fri, 10 Sep 2021 23:17:14 GMT
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.2.34
Content-Disposition: attachment; filename="date6"
Content-Length: 284816
Content-Type: application/octet-stream
```

```
tcpdump -nn -A -r angry-poutine.pcap 'tcp src port 80 and src host 194.62.42.206' |
grep -aE 'HTTP/1\.[01] |Date:|Server:|Content-'
```

- `filename="date6"` explique pourquoi les rapports publiés appellent le fichier `date6.dll` alors que l'URL finit par `date1`.
- L'en-tête `Date` indique 23:17:14, alors que le paquet est horodaté 23:17:28 : l'horloge du serveur retarde de 14 secondes. On se fie toujours à l'horodatage de la capture (le capteur), jamais à l'horloge d'un serveur malveillant. C'est la source de l'heure « 23:17:14 » de certains corrigés.
- `Server:` révèle un Apache/PHP vieillissant sur CentOS : typique d'un site légitime compromis et détourné pour héberger le maliciel.

Q10. Le corps commence par `MZ` (signature des exécutables Windows) et contient `This program cannot be run in DOS mode`.

```
tcpdump -nn -A -r angry-poutine.pcap 'tcp src port 80 and src host 194.62.42.206' |
grep -a -m1 -o 'This program cannot be run in DOS mode'
```

L'URL n'a pas d'extension et le serveur propose le nom `date6` sans extension : on se fie au contenu, jamais au nom.

D. Le compte utilisateur

Q11. hobart.gunnarsson, dans le domaine `ANGRYPOUTINE.COM`.

```
tcpdump -nn -r angry-poutine.pcap -w - 'tcp port 88' | strings -n 6 | sort | uniq -c
| sort -rn | head -40
```

Début de la sortie :

```
     38 ANGRYPOUTINE.COM
     15 krbtgt
```

```
     13 20370913024805Z
     11 hobart.gunnarsson
      6 ANGRYPOUTINE
```

Dans Kerberos, le nom du client et le domaine circulent en clair ; seuls les tickets et l'authentificateur sont chiffrés. `krbtgt` est le service qui délivre les tickets. `20370913024805Z` est la date d'expiration que Windows demande toujours par défaut (13 septembre 2037). Ici, le compte d'ordinateur n'apparaît pas comme client : le poste s'était authentifié avant le début de la capture. Piège à éviter : avec `tcpdump -A | grep`, les octets non imprimables deviennent des points et les noms se retrouvent collés ensemble.

E. Le trafic chiffré

Q12. 35 Client Hello vers ces trois adresses. Seul 34.117.59.81 envoie un SNI (`myexternalip.com`). Les 33 connexions vers 94.158.245.52 et celle vers 167.172.37.9 n'en ont pas : connexion directe par IP.

```
tcpdump -nn -A -r angry-poutine.pcap 'tcp dst port 443 and (tcp[((tcp[12:1] & 0xf0) >> 2):1] = 0x16) and (tcp[((tcp[12:1] & 0xf0) >> 2)+5:1] = 0x01) and (dst host 167.172.37.9 or dst host 94.158.245.52 or dst host 34.117.59.81)'
```

Explication du filtre, à faire au tableau :

1. `tcp[12:1] & 0xf0` lit les 4 bits de poids fort de l'octet 12 de l'en-tête TCP : la longueur de l'en-tête en mots de 32 bits.
2. `>> 2` décale de 2 au lieu de 4, ce qui revient à « décaler de 4 puis multiplier par 4 » : on obtient la longueur en octets, donc la position du début des données.
3. Le premier octet des données vaut `0x16` pour un enregistrement TLS de type Handshake.
4. L'octet 5 des données (après 5 octets d'en-tête d'enregistrement) vaut `0x01` pour un Client Hello.

Q13. `myexternalip.com`, résolu à 23:19:21.509 (c'est bien un nom de 16 caractères). C'est un service légitime qui renvoie l'IP publique de celui qui l'appelle. Un maliciel s'en sert pour connaître l'IP publique et le pays de sa victime. Le lien avec l'infection est fort : 2 minutes après, dans la même rafale que la reconnaissance. Ce n'est pas un IOC à bloquer aveuglément, mais un **indicateur de comportement** : un poste d'entreprise n'a aucune raison de l'appeler.

F. La reconnaissance

Q14. `DESKTOP-MUYEYV7.angrypoutine.com` → 10.9.10.103, résolu à 23:19:22. Canal nommé : `\PIPE\LANMAN`.

```
tcpdump -nn -r angry-poutine.pcap -w - 'host 10.9.10.103 and tcp port 139' | strings -n 5 -e l | sort | uniq -c | sort -rn
```

Sortie :

```
      2 \PIPE\LANMAN
      2 Windows 10 Pro 6.3
      2 Windows 10 Pro 19043
      2 ANGRYPOUTINE
      2  angrypoutine.com
      1 \\DESKTOP-MUYEYV7\IPC$
```

`strings -e l` cherche des chaînes en UTF-16 little-endian, l'encodage de Windows ; sans `-e l`, on ne voit presque rien. Le canal `\PIPE\LANMAN` sur le partage `IPC$` sert à l'appel `NetServerEnum2` : c'est la

commande **`net view`**, qui liste les postes du réseau. La réponse révèle même la version de Windows des postes (build 19043).

Q15. Groupes interrogés : **Domain Computers** et **domain admins**. Commandes : `net group "Domain Computers" /domain` et `net group "domain admins" /domain`.

```
tcpdump -nn -r angry-poutine.pcap -w - 'host 10.9.10.9 and tcp port 445' | strings -n 6 -e l | sort | uniq -c | sort -rn | head -30
```

La sortie mélange la reconnaissance et le trafic normal du domaine : les chemins `Policies\{31B2F340-...}` et `gpt.ini` sont les stratégies de groupe (GPO) lues à l'ouverture de session. Il faut savoir les écarter. Pourquoi un attaquant fait-il ça : savoir combien de postes compte le domaine et quels comptes sont administrateurs du domaine, donc quelles cibles viser pour se déplacer et prendre le contrôle. C'est la préparation classique d'un rançongiciel. Le détail des comptes obtenus est plus simple à lire dans Wireshark (étape 3, Q14).

G. Bilan

Q16. Confirmé : le téléchargement d'un exécutable, son domaine et son URL, l'identité du poste et de l'utilisateur, des IP de C2 codées en dur, une reconnaissance du domaine. Corrigé : l'hypothèse d'un lien DNS pour 167.172.37.9. Encore ouvert : le hash et la famille du maliciel, le certificat du C2, le contenu chiffré.

Q17. Extraire le fichier : il faudrait réassembler 212 segments TCP à la main (ou utiliser `tcpflow`, un autre outil). Les protocoles SMB, SAMR et Kerberos se lisent seulement à coups de `strings`, sans structure. C'est la transition vers Wireshark.

Étape 3 — Le décodage complet (Wireshark)

Conclusion attendue : le fichier téléchargé est un DLL Windows 64 bits de 284 816 octets, identifié par son hash comme BazarLoader. Les deux serveurs HTTPS suspects utilisent des certificats auto-signés générés ensemble. Deux minutes après l'infection, l'attaquant a obtenu la liste des 5 postes du domaine et appris que le seul administrateur du domaine est `Administrator`.

Repères de syntaxe Wireshark

- Un **filtre d'affichage** (barre du haut) n'est pas un filtre BPF : `ip.addr == 10.9.10.102`, `tcp.port == 443`, `&&`, `||`, `!`, `contains`, `==`.
- Un nom de champ seul (`http.request`) veut dire « le champ existe ».
- Clic droit sur un champ décodé > *Apply as Column* pour l'ajouter au tableau ; *Apply as Filter* pour filtrer dessus.
- View > Time Display Format > *UTC Date and Time of Day*.

A. Vue d'ensemble

Q1. Même classement qu'à l'étape 1 : 23.1.237.225, 23.1.237.216, 52.238.248.6, 10.9.10.9, 23.1.237.200, 167.172.37.9, 194.62.42.206, 94.158.245.52. Deux différences à expliquer : les octets sont un peu plus élevés parce que Wireshark compte la trame entière (en-tête Ethernet compris), alors que NetFlow commence à l'en-tête IP. Et le pcap contient 9 221 trames contre 8 959 paquets dans les flux : les 262 trames d'écart sont de l'ARP, qui n'est pas de l'IP et n'apparaît donc jamais dans NetFlow.

Q2. Entre autres : DHCP, DNS, NBNS, NTP, Kerberos, LDAP, SMB, SMB2, DCE/RPC, HTTP et TLS. Deux protocoles doivent attirer l'œil : **SAMR** (114 trames) et **LANMAN** (4 trames), des protocoles d'énumération (voir F). Piège de lecture : Protocol Hierarchy attribue les octets à la couche la plus haute *décodée dans chaque trame*. Les segments de données d'un gros téléchargement HTTP sont comptés sous `tcp`, pas sous `http`. On ne peut donc pas y lire directement quel protocole applicatif transporte le plus d'octets.

B. Identité complète

Q3. Filtre `dhcp` (sur les très anciennes versions de Wireshark : `bootp`). Option 12 *Host Name* = `DESKTOP-KKITB6Q`, *Client MAC address* = `00:4f:49:b1:e8:c3`. Bonus : le filtre `nbns` montre les noms NetBIOS décodés, ce que tcpdump 4.99 ne fait pas.

Q4. hobart.gunnarsson (11 trames Kerberos).

```
kerberos.CNameString && !(kerberos.CNameString contains "$")
```

`kerberos.CNameString` est le nom du client dans les échanges Kerberos. Les comptes d'ordinateur finissent par un dollar, d'où l'exclusion. Dans ce pcap, aucun compte d'ordinateur n'apparaît (le poste s'était authentifié avant la capture), mais le filtre est le bon réflexe : dans une capture d'entreprise, ils sont très nombreux.

Q5. `ANGRYPOUTINE.COM` (champ `kerberos.realm`), et parfois la forme courte NetBIOS `ANGRYPOUTINE`. Le realm Kerberos est le nom DNS du domaine AD en majuscules.

C. Le téléchargement

Q6. `http://simpsonsavingss.com/bmdff/BhoHsCtZ/MLdmpfjaX/5uFG3Dz7yt/date1?BNLv65=pAAS`, demandée à 23:17:27.822.

```
http.request
```

Le champ *Full request URI* (`http.request.full_uri`) assemble `Host` et chemin ; on peut l'ajouter en colonne. Le filtre affiche 58 requêtes, dont beaucoup de SSDP (239.255.255.250, découverte de périphériques) et de Windows Update : une seule vers un domaine inconnu.

Q7. Requête et réponse réassemblées dans une seule fenêtre, en couleurs (client en rouge, serveur en bleu), avec le corps binaire complet : `MZ`, le message DOS, puis le reste de l'exécutable. Wireshark applique aussi le filtre `tcp.stream eq N`, qui isole cette connexion.

Q8. Environ 46 objets (le compte exact peut varier d'une version de Wireshark à l'autre) :

Objets	Source	Verdict
33 fichiers `.cab`	download.windowsupdate.com	Mises à jour Windows, légitimes
`am_delta_patch_1.349.439.0_...exe` (en 5 morceaux)	au.download.windowsupdate.com	**Exécutable légitime** : mise à jour des définitions de Microsoft Defender
6 images PNG/JPEG	store-images.s-microsoft.com	Icônes du Microsoft Store
`ncsi.txt`	www.msftncsi.com	Test de connectivité Windows

Objets	Source	Verdict
`date1?BNLv65=pAAS` (284 816 octets)	simpsonsavingss.com	**Malveillant**

C'est le piège voulu : deux exécutables Windows, un seul malveillant. On les distingue par le domaine source (Microsoft contre un domaine inconnu), le contexte, puis le hash (à chercher dans VirusTotal) et la signature numérique (celui de Microsoft est signé). Sauvegarder l'objet suspect dans la VM, dans un dossier dédié, sans l'ouvrir.

Q9. SHA256 = `eed363fc4af7a9070d69340592dcab7c78db4f90710357de29e3b624aa957cf8`. `file` : `PE32+ executable (DLL) (GUI) x86-64, for MS Windows, 4 sections`, soit un DLL Windows 64 bits.

```
sha256sum date1*
file date1*
```

À la sauvegarde, Wireshark peut remplacer le `?` par `%3f` (`date1%3fBNLv65=pAAS`). Le motif `date1*` couvre les deux cas ; sinon, mettre le nom entre apostrophes pour que le shell n'interprète pas le `?`.

D. Le trafic chiffré

Q10. Les 33 connexions vers 94.158.245.52 et celle vers 167.172.37.9 n'ont **aucun SNI**. Celle vers 34.117.59.81 en a un (`myexternalip.com`). Un navigateur envoie presque toujours un SNI ; une connexion TLS sans SNI vers une IP brute est typique d'un maliciel dont l'adresse du C2 est codée en dur.

```
tls.handshake.type == 1
tls.handshake.type == 1 && !tls.handshake.extensions_server_name
```

`tls.handshake.type == 1` = Client Hello. Ajouter `tls.handshake.extensions_server_name` en colonne. Le deuxième filtre ne garde que les Client Hello sans SNI : il ne reste que les 34 connexions suspectes.

Q11. Certificat de 94.158.245.52, en TLS 1.2 :

Champ	Valeur
Émetteur (issuer)	CN=londonareloeli.uk, OU=UK System, O=LL EST, L=London, ST=London, C=UK
Sujet (subject)	Identique à l'émetteur
Validité	Du 2021-09-08 08:03:21 au 2022-09-08 08:03:21 (UTC)
Numéro de série	0fa40f28dc0d219a4d9db7b06b084814ef93c8f0

```
tls.handshake.type == 11 && (ip.src == 94.158.245.52 || ip.src == 167.172.37.9)
```

`type == 11` = message *Certificate*. Déplier *Certificate* > *signedCertificate* > *issuer*, *subject*, *validity*.

- **Auto-signé** : émetteur = sujet, aucune autorité reconnue. Aucun service légitime n'exposerait ça à un poste client.
- **Créé deux jours avant l'infection** (8 septembre) : infrastructure montée pour la campagne.
- **`C=UK` n'est pas un code pays valide** : le code ISO du Royaume-Uni est `GB`. Signe d'un certificat généré à la va-vite par un script.

- **167.172.37.9** présente un certificat avec un numéro de série différent, mais exactement les mêmes champs et les mêmes dates, à la seconde près. Conclusion : les deux certificats ont été générés par le même outil au même moment, donc par le même opérateur. On peut chercher ces champs dans des moteurs comme Censys ou Shodan pour trouver d'autres serveurs de la même infrastructure (« pivot »).
- **Seulement 11 des 33 sessions** montrent un certificat : les 22 autres reprennent une session TLS existante (*session resumption*). Cette poignée de main abrégée saute le message *Certificate*.

E. Visualiser le beaconing

Q12. Des pics isolés qui vont par deux et reviennent toutes les 3 à 8 minutes, sur un fond plat : le même motif que les intervalles calculés à l'étape 1, Q11. Graphe : Statistics > I/O Graphs, intervalle 10 s, filtre `ip.addr == 94.158.245.52`.

F. La reconnaissance

Q13.

Heure (UTC)	Filtre	Ce qu'on voit	Commande Windows
23:19:21	`tls.handshake.extensions_server_name contains "myexternalip"`	Connexion HTTPS à myexternalip.com	Vérification de l'IP publique
23:19:24–25	`lanman`	`NetServerEnum2` vers DESKTOP-MUYEYV7 (10.9.10.103)	`net view`
23:19:26–27	`samr`	`LookupNames` « Domain Computers » puis « domain admins », `QueryGroupMember`, `LookupRids` vers le DC	`net group "Domain Computers" /domain` et `net group "domain admins" /domain`

Ajouter `samr.samr_LookupNames.names` en colonne pour lire les noms demandés. Attention : `samr` montre aussi un `LookupNames` de `hobart.gunnarsson` à 23:16:56, pendant l'ouverture de session. C'est du trafic Windows normal : c'est le moment (2 minutes après l'infection, en pleine rafale) et les noms demandés qui rendent la séquence de 23:19 suspecte.

Q14. Les réponses `LookupRids` (filtre `samr.opnum == 18`) donnent :

- **5 postes du domaine** : `DESKTOP-TB2QUMK$`, `DESKTOP-GOJ2LXH$`, `DESKTOP-KKITB6Q$`, `DESKTOP-MUYEYV7$`, `DESKTOP-SRA7Q54$`.
- **1 administrateur du domaine** : `Administrator`. C'est le compte le plus précieux pour l'attaquant : avec lui, il contrôle tout le domaine.

La session vers DESKTOP-MUYEYV7 a été ouverte **sans compte** : le Session Setup montre `NTLMSSP_AUTH, User: \`, une session anonyme (*null session*). C'est ainsi que fonctionne `net view` sur ce service. Le fait que le poste réponde à une session anonyme est un point de durcissement à discuter.

G. Bilan

Q15. BazarLoader (famille Bazar, campagne TA551). On cherche le **hash** dans VirusTotal ou MalwareBazaar : on ne téléverse pas le fichier. En entreprise, un téléversement rend le fichier accessible à d'autres et peut révéler à l'attaquant qu'on l'a trouvé.

Q16. Le décodage par protocole (Kerberos, TLS, SAMR, LANMAN), l'extraction d'un fichier en deux clics, le réassemblage TCP, les champs en colonnes, les statistiques et la visualisation dans le temps. Ce que Wireshark ne remplace pas : la vue d'ensemble rapide des flux sur des heures ou des jours de trafic.

Synthèse — Rapport type et retour

A. Rapport d'incident type

Résumé exécutif. Le 10 septembre 2021 à 23:17 UTC, le poste DESKTOP-KKITB6Q, utilisé par hobart.gunnarsson, a téléchargé et exécuté le maliciel BazarLoader, qui a installé une porte dérobée contrôlée à distance. Deux minutes plus tard, l'attaquant a dressé la liste des 5 postes du domaine et identifié le compte administrateur du domaine. Ce sont les préparatifs typiques d'un rançongiciel : le poste doit être isolé immédiatement et le compte Administrator surveillé.

Poste infecté

Champ	Valeur
Adresse IP	10.9.10.102
Adresse MAC	00:4f:49:b1:e8:c3
Nom d'hôte	DESKTOP-KKITB6Q (DESKTOP-KKITB6Q.angrypoutine.com)
Compte utilisateur	hobart.gunnarsson (domaine ANGRYPOUTINE.COM)
Système	Windows 10, 64 bits

Chronologie

Heure (UTC)	Événement
23:15:07	Le poste obtient son adresse par DHCP
23:16:55	Ouverture de session de hobart.gunnarsson (Kerberos)
23:17:01	Rafale de connexions Microsoft, compatible avec le démarrage d'Office
23:17:27	Résolution de simpsonsavingss.com et téléchargement de BazarLoader (284 816 octets)
23:17:31	Connexion HTTPS à 167.172.37.9 (charge suivante, environ 300 Ko)
23:17:36	Premier contact avec le C2 94.158.245.52
23:19:21	Vérification de l'IP publique (myexternalip.com)
23:19:24	Énumération des postes (`net view` vers DESKTOP-MUYEYV7)

Heure (UTC)	Événement
23:19:26	Énumération des groupes Domain Computers et domain admins (SAMR)
23:17 à 00:30	Beaconing C2 toutes les 3 à 8 minutes, jusqu'à la fin de la capture

Indicateurs de compromission (neutralisés pour éviter les clics)

Type	Valeur	Rôle
Domaine	simpsonsavingss[.]com	Serveur de téléchargement (site probablement compromis)
IP:port	194.62.42[.]206:80	Téléchargement de BazarLoader
URL	hxxp://simpsonsavingss[.]com/bmdff/BhoHsCtZ/MLdmpfjaX/5uFG3Dz7yt/date1?BNLv65=pAAS	Fichier malveillant
IP:port	167.172.37[.]9:443	Récupération de la charge suivante
IP:port	94.158.245[.]52:443	Serveur de commande et contrôle
Certificat TLS	Auto-signé, CN=londonareloeli[.]uk, O=LL EST, OU=UK System, C=UK	Infrastructure de l'attaquant (les deux IP en 443)
SHA256	eed363fc4af7a9070d69340592dcab7c78db4f90710357de29e3b624aa957cf8	DLL BazarLoader, 284 816 octets

Recommandations attendues (trois parmi celles-ci)

- Isoler le poste du réseau, puis le réinstaller plutôt que de le « nettoyer ».
- Réinitialiser le mot de passe de hobart.gunnarsson et révoquer ses sessions.
- Surveiller le compte Administrator et changer son mot de passe : c'est la prochaine cible.
- Bloquer les IOC au pare-feu (les IP, puisque le C2 n'utilise pas le DNS), au proxy et au DNS.
- Chercher les mêmes IOC sur les 4 autres postes du domaine (journaux du proxy, NetFlow, EDR).
- Revoir la politique de macros Office, la sensibilisation au hameçonnage et l'accès anonyme (null session) aux postes.

B. Tableau réflexif rempli

Information	Flux (nfdump)	Paquets (tcpdump)	Décodage (Wireshark)
IP du poste infecté	Première fois	Oui	Oui
IP du serveur de téléchargement	Première fois	Oui	Oui
Domaine du téléchargement	Indice (longueur du nom)	Première fois	Oui
URL complète	Non	Première fois	Oui

Information	Flux (nfdump)	Paquets (tcpdump)	Décodage (Wireshark)
Nature du fichier (exécutable)	Indice (volume)	Première fois	Oui
Hash du fichier	Non	Non (en pratique)	Première fois
Nom d'hôte	Non	Première fois	Oui
Compte utilisateur	Non	Première fois (avec `strings`)	Oui
IP du C2	Première fois	Oui	Oui
Périodicité du C2	Première fois	Possible	Oui (graphe)
Certificat du C2	Non	Partiel (`strings` montre le CN)	Première fois (complet)
Reconnaissance (postes et groupes énumérés)	Indice (rafale SMB/139 à 23:19)	Première fois (avec `strings -e l`)	Oui, détaillée

Message à faire ressortir : les flux trouvent le **quoi** et le **quand** (qui parle à qui, combien, à quel rythme) ; les paquets donnent le **qui** et le **comment**. En production, on a rarement le pcap complet, mais presque toujours des flux.

C. Pour aller plus loin

Q6. Règle de détection NetFlow — réponse type, à discuter :

- Pour chaque paire (poste interne, IP externe) sur une heure, compter les sessions TCP.
- Alerter si on voit au moins 10 sessions, des intervalles réguliers (écart-type faible par rapport à la moyenne), peu d'octets par session (moins de 10 Ko) et une IP absente de l'historique du réseau et des grands fournisseurs (Microsoft, Akamai, Google).
- Variante : un premier contact avec une IP inconnue qui renvoie plus de 200 Ko en HTTP, suivi en moins d'une minute d'un contact avec une autre IP inconnue.
- En combinant avec d'autres sources : une connexion TLS vers une IP qui n'a jamais été résolue par le DNS de l'entreprise, ou un poste qui appelle un service d'IP publique.

Intérêt pédagogique : discuter des faux positifs (mises à jour, télémétrie, messagerie) et de la fenêtre de temps.

Q7. Avant le téléchargement — réponse attendue : l'ouverture d'un document Office piégé reçu par courriel, le mode opératoire habituel de la campagne TA551. Ce qui l'appuie : la rafale de connexions Microsoft à 23:17:01 et la résolution de `office15client.microsoft.com` à 23:17:20, compatibles avec le démarrage d'Office, environ 26 secondes avant le téléchargement. Le User-Agent (composant Windows 32 bits, pas un navigateur) indique que le téléchargement a été lancé par un programme ou un script, pas par l'utilisateur dans son navigateur. Ce qui manque : le courriel et le document eux-mêmes, absents de la capture. Accepter toute hypothèse cohérente qui distingue clairement ce qui est observé de ce qui est supposé.

Sources

- BazarLoader traffic analysis — AngryPoutine (DEV Community) : famille, IOC, nom et hash du DLL, identité du poste.
- Angry Poutine — Traffic Analysis, Brian Biddle : URL complète, filtres Wireshark, User-Agent Windows NT 10.0.
- Exercice original sur malware-traffic-analysis.net : pcap et corrigé officiel (page non consultée directement pour ce corrigé).
- Toutes les valeurs du corrigé : vérifiées sur le pcap de l'exercice avec nfdump 1.7.10, tcpdump 4.99.4 et tshark 4.2.2.